# Two Pointer

---

## 1. What is Two Pointers Technique?

**Two pointers** ek technique hai jisme hum **array ya list par ek hi time par do indices (pointers)** use karte hai instead of one.

Usually:

- ek pointer start se
- ek pointer end se
  ya
- dono start se different speed pe

---

**Simple Definition**

"Two pointers technique uses two indices to traverse data structure efficiently, reducing time complexity by avoiding nested loops."

---

## 2. Why do we use Two Pointers?

Normal brute force:

Nested loops → $O(n^2)$

Two pointers se:

Single traversal → $O(n)$

👉 **Major benefit:** Time complexity reduce hoti hai

---

## 3. When to Use Two Pointers?

Use Two Pointers when:

✔ Array sorted ho
✔ Pair / triplet find karna ho
✔ Target sum problem ho
✔ Reverse / palindrome check
✔ Removing duplicates
✔ Window type problems

---

## 4. Types of Two Pointer Approaches

### ◇ Type 1: Opposite Direction (Start & End)

left → 0
right → n-1

Use when:

- Sorted array
- Sum find karna
- Palindrome

---

### ◇ Type 2: Same Direction (Fast & Slow Pointer)

slow → 0
fast → 0

Use when:

- Duplicates remove
- In-place modification
- Sliding window

---

## 5. Core Logic Pattern

### ✯ Pattern 1: Sum equals target (Sorted Array)

```
left = 0
right = n - 1

while(left < right):
   sum = arr[left] + arr[right]

   if(sum == target):
      return found

   else if(sum < target):
      left++

   else:
      right--
```

👉 Time Complexity: **O(n)**

---

🌟 **Pattern 2: Remove duplicates (Slow/Fast)**

```
slow = 0

for fast in range(1 to n):
   if arr[fast] != arr[slow]:
      slow++
      arr[slow] = arr[fast]
```

👉 Unique elements at beginning

---

🌟 **Pattern 3: Palindrome Check**

```
left = 0
right = length - 1

while(left < right):
   if s[left] != s[right]:
      return false
   left++
   right--
```

---

**6. Classic Problems Using Two Pointers**

✔ **Problem 1: Two Sum (Sorted Array)**

👉 Find pair with target sum

Time: **O(n)**

---

✔ **Problem 2: Remove Duplicates from Sorted Array**

👉 Use slow & fast pointer

---

✔ **Problem 3: Reverse Array / String**

👉 Swap left & right

---

✔ **Problem 4: Container With Most Water**

👉 Move smaller height pointer

---

✔ **Problem 5: 3 Sum Problem**

👉 Sort + Two pointer inside loop

Time: **$O(n^2)$**

---

### 7. Optimization Comparison

| Approach | Time Complexity |
|---|---|
| | |
| Brute Force | $O(n^2)$ |
| Two Pointers | $O(n)$ |

---

### 8. Important Interview Tricks 💡

🔥 **Trick 1: Sorted array dikhe → think Two Pointer**

---

🔥 **Trick 2: Pair/triplet sum → use left & right**

---

🔥 **Trick 3: Duplicate remove → slow-fast pointer**

---

🔥 **Trick 4: Window problems → sliding window (two pointers)**

---

### 9. Common Mistakes ❌

❌ Array sorted nahi hai but two pointer use karna
❌ pointer move galat direction me
❌ edge cases ignore karna

❌ left < right condition bhoolna
❌ duplicates skip nahi karna (3 sum)

---

### 10. Time Complexity Analysis

Two pointer me generally:

👉 Each element max **1–2 times visit hota hai**

So complexity = **O(n)**

---

### 11. Dry Run Example

Array:

[1, 2, 3, 4, 6]
target = 6

Steps:

left = 1 (index 0)
right = 6 (index 4)

1+6=7 > target → right--
1+4=5 < target → left++
2+4=6 = found ☑

---

### 12. When NOT to Use Two Pointers

❌ Unsorted data without hashing
❌ Random access not allowed (linked list except special cases)
❌ Need all combinations (use recursion)

---

### 13. Golden Rules

✔ Mostly sorted arrays me use hota hai
✔ Nested loops ko replace karta hai
✔ Left-right movement based on condition
✔ Time complexity reduce karta hai
✔ Interview me highly asked pattern

## INTERVIEW FAQs – TWO POINTERS

### ❓ 1. What is Two Pointer Technique?

👉 Using two indices to traverse array efficiently

### ❓ 2. Why use it?

👉 Reduce **$O(n^2) \rightarrow O(n)$**

### ❓ 3. When to use it?

✔ Sorted array
✔ Pair sum
✔ Reverse
✔ Palindrome
✔ Sliding window

### ❓ 4. Two pointers vs Hashing?

| Technique | Use |
|---|---|
| Two pointers | Sorted array |
| Hashing | Unsorted array |

### ❓ 5. Fast & Slow pointer use cases?

✔ Cycle detection
✔ Remove duplicates
✔ Middle of linked list

### ❓ 6. Time complexity of Two Pointers?

👉 Usually **O(n)**

---

❓ **7. Space complexity?**

👉 **O(1)** (in-place)

---

❓ **8. Can we use two pointers in strings?**

👉 YES (palindrome, reverse, substring)

---

❓ **9. Difference: Sliding Window vs Two Pointer?**

👉 Sliding window = dynamic range
👉 Two pointer = fixed movement logic

---

❓ **10. Common problems asked in interviews**

✔ Two Sum sorted
✔ 3 Sum
✔ Container With Most Water
✔ Remove duplicates
✔ Valid palindrome